

String Matching with Finite Automata

1 Core Idea

What the Automaton Stores

For pattern $P[1 : m]$, state q means:

the first q characters of P match the current text suffix.

State m means the whole pattern has matched.

The automaton scans the text once. It never moves backward in the text. All fallback decisions are already stored in the transition table.

States

$$Q = \{0, 1, 2, \dots, m\}, \quad q_0 = 0, \quad A = \{m\}.$$

2 Suffix Function

Definition

Let $P_k = P[1 : k]$. Define

$$\sigma(x) = \max\{k \mid P_k \text{ is a suffix of } x\}.$$

So $\sigma(x)$ is the length of the longest prefix of P that appears at the end of x .

Example with $P = \mathbf{ab}$	Longest prefix of P at the end	$\sigma(x)$
$x = \varepsilon$	none	0
$x = \mathbf{ccaca}$	a	1
$x = \mathbf{ccab}$	ab	2

Match Test

$$\sigma(T_i) = m \iff P \text{ ends at position } i.$$

3 Transition Function

Transition Rule

If the current state is q and the next character is a , then

$$\delta(q, a) = \sigma(P_q a).$$

Meaning: append the new character to the currently matched prefix, then keep the longest prefix of P that is still a suffix.

Case	What happens
Next character matches	If $a = P[q + 1]$, then $\delta(q, a) = q + 1$.
Mismatch	Fall back to the largest smaller prefix of P that is still a suffix.

4 Key Lemmas

Lemma 1 (One Character Can Increase the Match by At Most One). *For any string x and character a ,*

$$\sigma(xa) \leq \sigma(x) + 1.$$

Proof. If $\sigma(xa) = r$, then P_r is a suffix of xa . Remove the last character from both strings. Then P_{r-1} is a suffix of x . Therefore

$$r - 1 \leq \sigma(x),$$

so $r \leq \sigma(x) + 1$. □

Lemma 2 (State q Is Enough Information). *If $q = \sigma(x)$, then for any character a ,*

$$\sigma(xa) = \sigma(P_q a).$$

Proof. State q says that the useful suffix of x is exactly P_q . After reading a , any new match must be a suffix of $P_q a$. Therefore the next state can be computed from $P_q a$, not from the full old text x . □

5 Matcher Algorithm

Finite-Automaton Matcher

```
FINITE-AUTOMATON-MATCHER(T, delta, m)
1  q = 0
2  for i = 1 to n
3      q = delta(q, T[i])
4      if q == m
5          print "Pattern occurs with shift", i - m
```

Why the Shift is $i - m$

When state m is reached at text position i , the pattern occupies positions

$$i - m + 1, \dots, i.$$

So the 0-based shift is $i - m$.

6 Correctness

Theorem 1 (Main Invariant). *After reading T_i , the current automaton state is*

$$\phi(T_i) = \sigma(T_i).$$

Proof. Use induction on i . Initially, $T_0 = \varepsilon$, so the state is $0 = \sigma(\varepsilon)$.

Assume after reading T_i , the state is $q = \sigma(T_i)$. Let the next character be $a = T[i + 1]$. The automaton moves to

$$\delta(q, a) = \sigma(P_q a).$$

By Lemma 2,

$$\sigma(P_q a) = \sigma(T_i a) = \sigma(T_{i+1}).$$

So the invariant remains true. □

Correctness Result

The matcher reports exactly when $q = m$. By the invariant, this is exactly when $\sigma(T_i) = m$, meaning P is a suffix of T_i . Therefore every reported shift is correct.

7 Building the Transition Table

Direct Construction

```

COMPUTE-TRANSITION-FUNCTION(P, Sigma)
1  m = P.length
2  for q = 0 to m
3      for each character a in Sigma
4          k = min(m+1, q+2)
5          repeat
6              k = k - 1
7              until P[1:k] is a suffix of P[1:q]a
8          delta(q,a) = k
9  return delta
    
```

Manual Rule

To compute $\delta(q, a)$: write $P_q a$, then choose the longest prefix of P that is a suffix of that string.

8 Worked Example

Let

$$P = ab, \quad \Sigma = \{a, b, c\}.$$

The states are 0, 1, 2.

State q	read a	read b	read c
0	1	0	0
1	1	2	0
2	1	0	0

Why $\delta(2, \mathbf{a}) = 1$: after **ab** then **a**, we have **aba**. The longest prefix of **ab** that is a suffix of **aba** is **a**.
Scan $T = \text{ccab}$:

i	$T[i]$	State q	Meaning
1	c	0	no useful match
2	c	0	no useful match
3	a	1	matched a
4	b	2	matched ab

At $i = 4$, state $2 = m$. Therefore the match has shift

$$i - m = 4 - 2 = 2.$$

9 Complexity

Part	Time
Build transition table directly	$O(m^3 \Sigma)$
Build transition table with improved method	$O(m \Sigma)$
Scan text after table is built	$\Theta(n)$

Important

The matching phase is always linear. The expensive part is preprocessing the transition table.

10 Final Summary

One-Box Summary

State q means $P[1 : q]$ matches the current text suffix.
 $\sigma(x) = \max\{k \mid P[1 : k] \text{ is a suffix of } x\}$.
 $\delta(q, a) = \sigma(P_q a)$.
 $\phi(T_i) = \sigma(T_i)$ is the main invariant.
 $q = m \Rightarrow$ pattern found with shift $i - m$.
 Matching time after preprocessing is $\Theta(n)$.